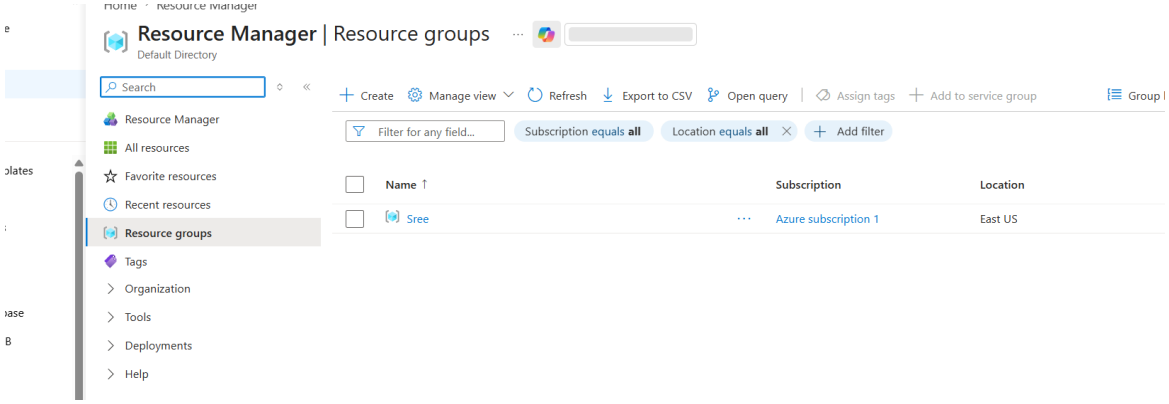
Incremental Data Load from On-Premises MySQL

to Azure SQL Database using Azure Data Factory (ADF)

This document walks through implementing an incremental (delta) load pattern from an on-premises MySQL database into Azure SQL Database using Azure Data Factory. Instead of a plain insert-only copy, the pipeline uses an Upsert write behavior keyed on a business column (OrderID), so existing records are updated and new records are inserted in the same run — the foundation of an incremental load.

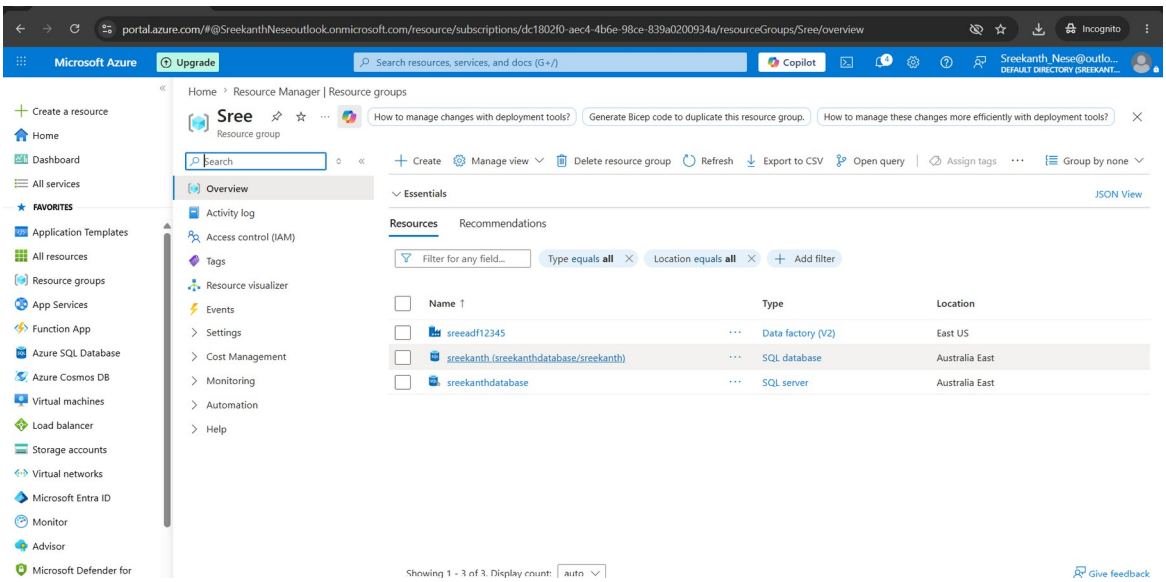
Step 1: Created a Resource Group

A Resource Group named “sree” was created in Azure to act as the logical container for all resources used in this project — keeping the Data Factory, SQL Server, and SQL Database organized under a single, manageable unit.



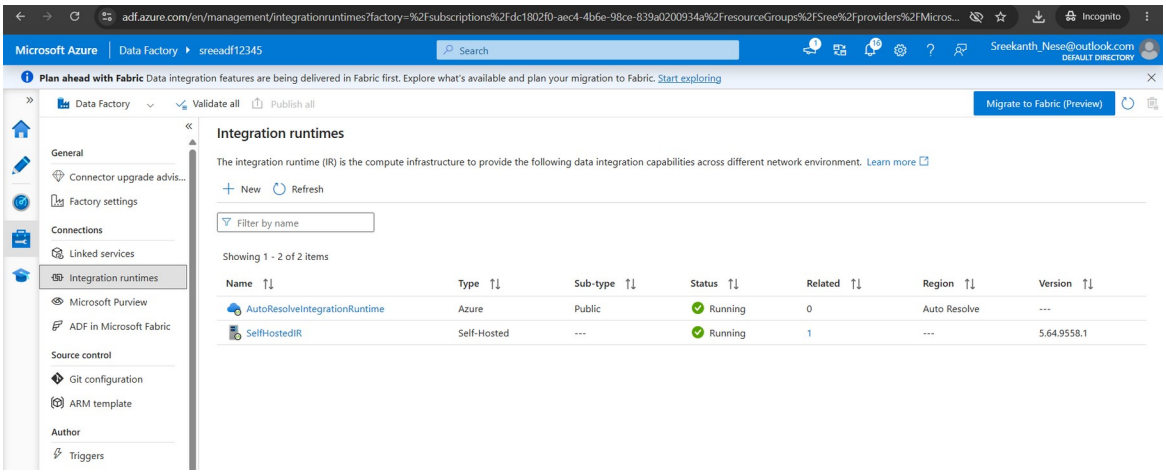
Step 2: Provisioned Core Resources — ADF, SQL Server, and SQL Database

Inside the resource group, three resources were provisioned: an Azure Data Factory (sreeadf12345) to orchestrate the pipeline, an Azure SQL Server (sreekanthdatabase), and an Azure SQL Database (sreekanth) to serve as the destination for the incrementally loaded data.



Step 3: Created a Self-Hosted Integration Runtime

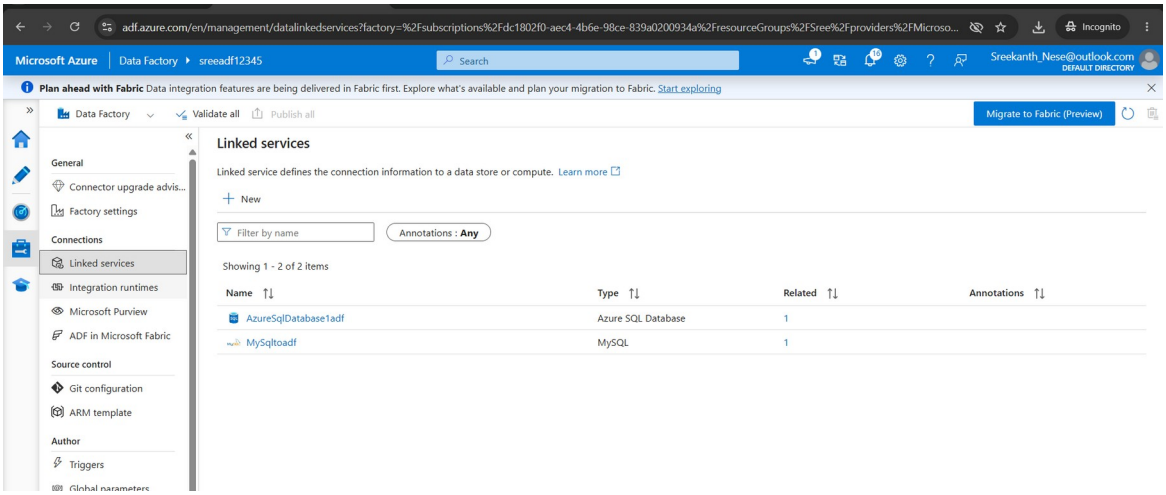
Since the source MySQL database sits on-premises and isn't reachable directly from the cloud, a Self-Hosted Integration Runtime (SelfHostedIR) was set up alongside the default AutoResolveIntegrationRuntime. It runs on a local machine and securely bridges ADF to the on-prem network so the pipeline can read from MySQL.



Step 4: Created Linked Services

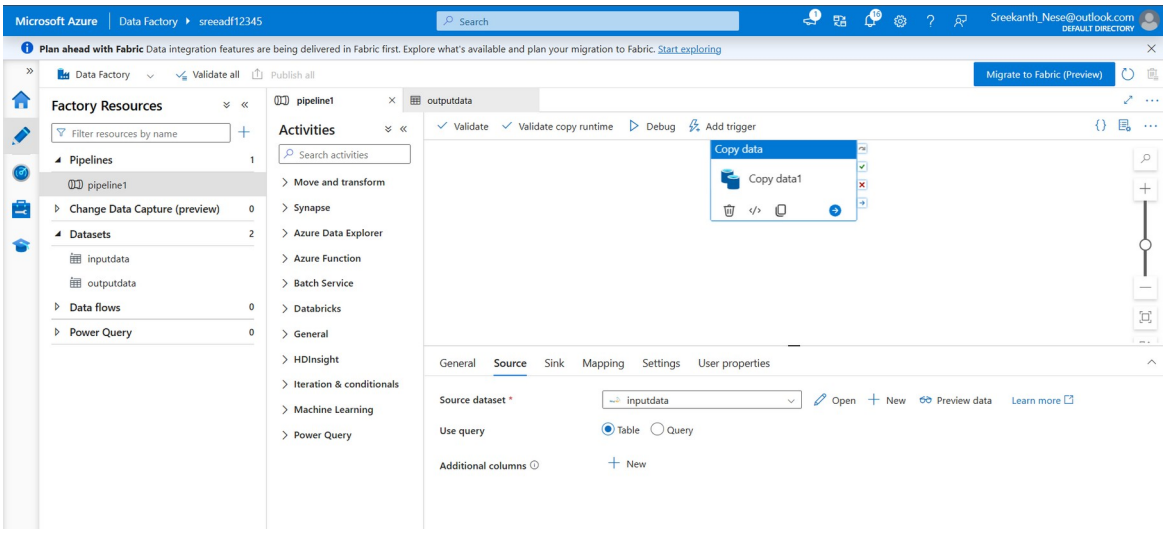
Two linked services were configured to define how ADF connects to each data store:

- MySqltoadf — connects to the on-premises MySQL database via the Self-Hosted Integration Runtime
- AzureSqlDatabase1adf — connects to the Azure SQL Database (the incremental load destination)



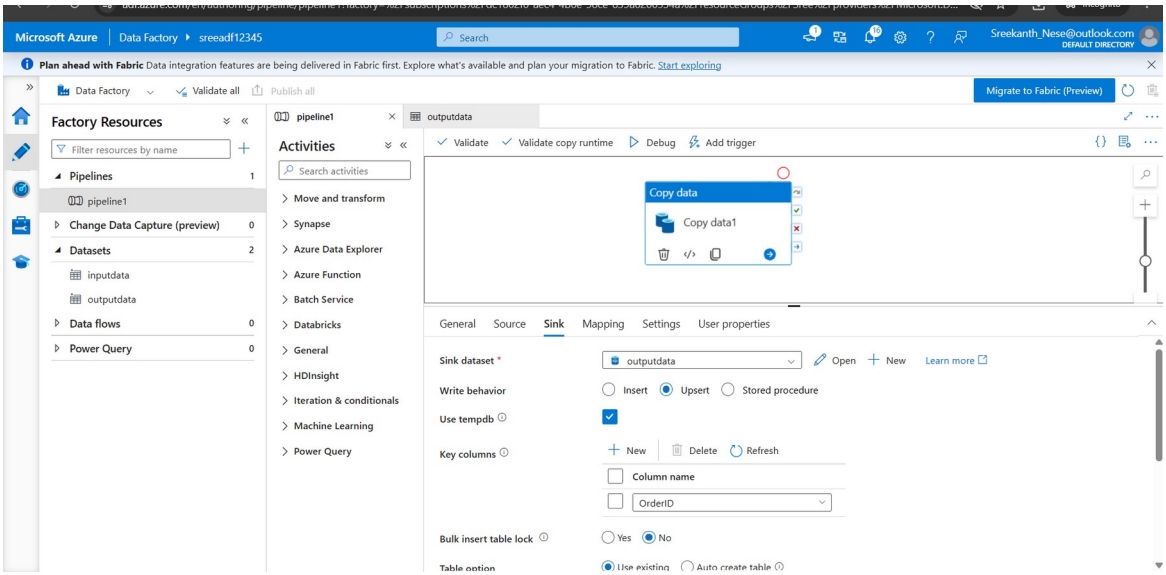
Step 5: Configured the Copy Activity — Source

A pipeline was created with a Copy Data activity. On the Source tab, the dataset was set to “inputdata,” pointing at the source table in the on-prem MySQL database that supplies the records to be loaded.



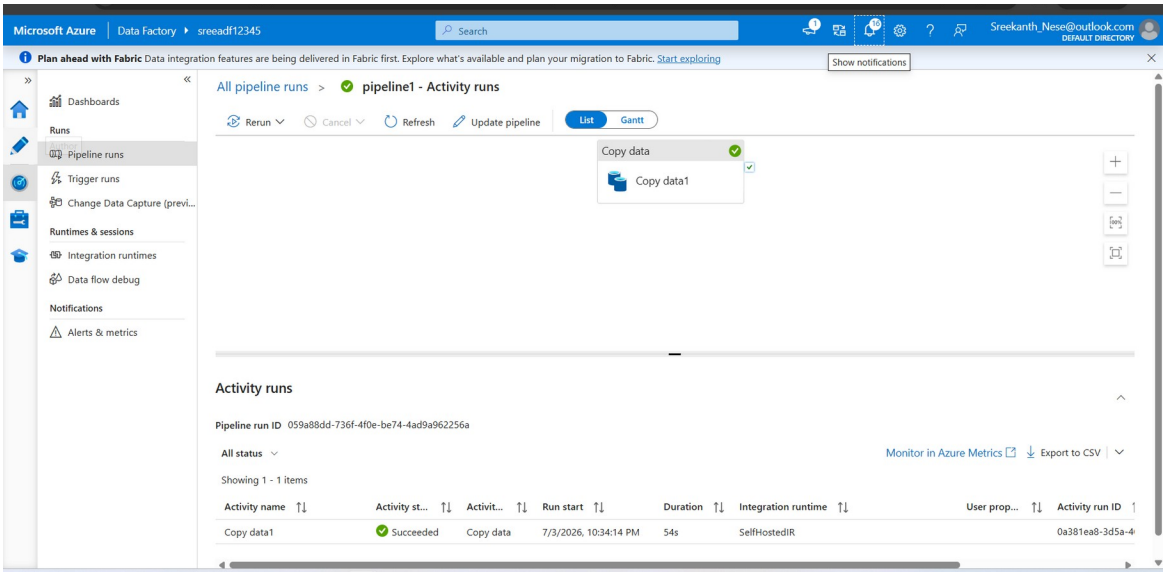
Step 6: Configured the Copy Activity — Sink (Upsert Logic)

On the Sink tab, the destination dataset was set to “outputdata” (Azure SQL Database), and the Write behavior was set to Upsert instead of a plain Insert. OrderID was set as the key column, which tells ADF how to match incoming rows against existing ones: matching rows are updated, and new OrderIDs are inserted. The Auto create table option was also enabled for the target table.



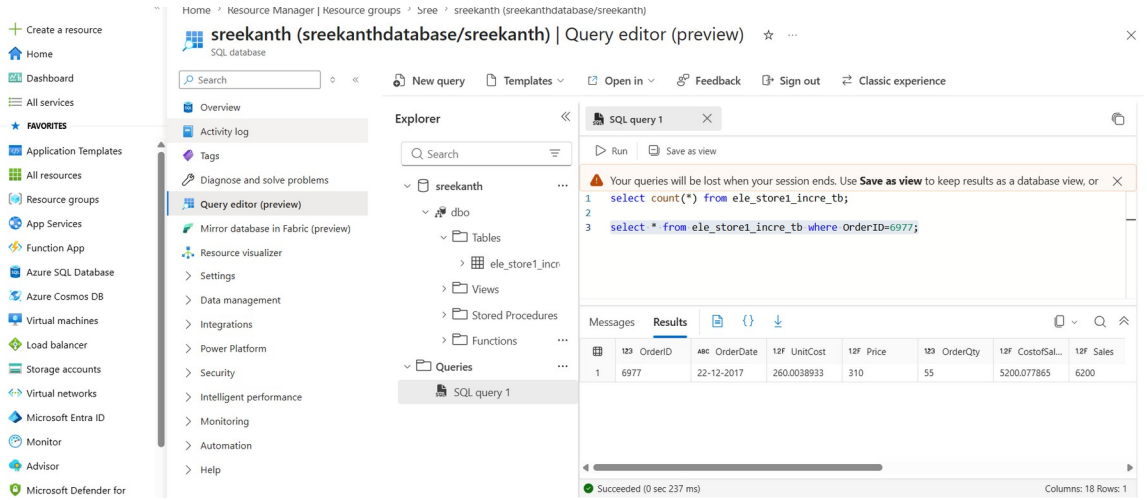
Step 7: Ran the Pipeline and Validated the Activity Run

The pipeline was executed, and the Copy Data activity run was monitored in the ADF monitoring tab. The activity completed with a Succeeded status in 54 seconds, running on the SelfHostedIR — confirming the on-prem-to-cloud connection and the upsert copy worked as expected.



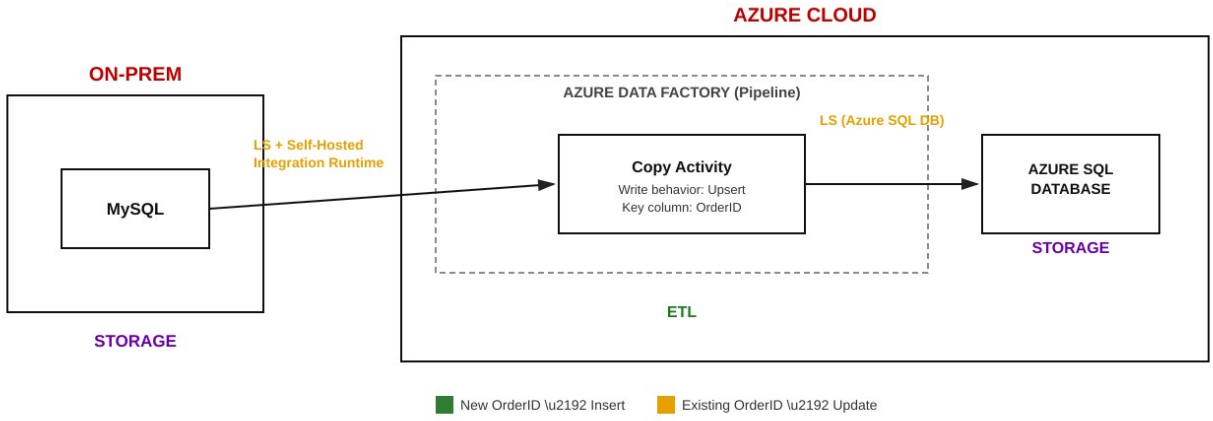
Step 8: Validated the Data in Azure SQL Database

Finally, the loaded data was verified directly in Azure SQL Database using the built-in Query Editor. A row count query confirmed the records landed in the ele_store1_incre_tb table, and a targeted lookup on a specific OrderID confirmed the upsert logic correctly matched and returned the right record — validating that the incremental load behaved as designed.



Architecture Diagram

The diagram below shows the end-to-end flow: the on-prem MySQL database is reached through the Self-Hosted Integration Runtime, and a single Copy Activity in ADF handles the incremental load — using Upsert as the write behavior with OrderID as the key column — before landing the data in Azure SQL Database.



How the Incremental Load Works

The illustration below shows what happens on each run: rows with an OrderID that already exists in the target get their values updated (e.g. OrderID 6976's price change), rows with a brand-new OrderID get inserted (e.g. OrderID 6978), and rows that haven't changed are left untouched — so every run only moves what's actually new or different.

How the Incremental (Upsert) Load Works

Source (MySQL) - Latest Snapshot

OrderID	Price	Status
6975	120	unchanged
6976	310	price updated
6977	310	unchanged
6978	205	new row
6979	95	unchanged

ADF Copy (Upsert on OrderID)

Target (Azure SQL) - After Upsert

OrderID	Price	Action Taken
6975	120	no change
6976	310	UPDATE
6977	310	no change
6978	205	INSERT
6979	95	no change

- New OrderID in source \u2192 INSERT into target
- Existing OrderID with changed value \u2192 UPDATE in target
- Unchanged OrderID \u2192 left as-is (no action)

Key column used for matching: OrderID | Write behavior: Upsert

Key Takeaway

Switching the Copy activity's write behavior from Insert to Upsert, with a business key column (OrderID), turns a simple one-time copy into a true incremental load pattern — new records get inserted and existing records get refreshed automatically on every pipeline run, without duplicating rows or requiring a full reload of the source table.